

2. DHCP : Analyse d'un échange DORA

vous trouverez ici une trace Wireshark capturant un échange d'allocation d'adresse IP entre un client et un serveur DHCP.

Cette trace ne contient que des échanges DHCP, mais gardez en tête que pour filtrer le DHCP dans Wireshark, il faut utiliser le filtre "bootp" (pourquoi?).

Analysez la trace pour répondre aux questions suivantes :

Quelles sont les ports utilisés ?

S'agit-il de tcp ou udp ?

DHCP est une extension de BOOTP et utilise exactement les mêmes champs et numéros de ports (UDP 67/68). Wireshark regroupe donc BOOTP et DHCP sous le même filtre bootp.

En pratique : bootp capture tous les messages DHCP (DISCOVER/OFFER/REQUEST/ACK, etc.).

Pour chacun des 4 échanges, indiquez :

L'adresse IP source et l'adresse IP destination

L'adresse MAC source et l'adresse MAC destination

N°	IP source	IP destination	MAC source	MAC des
1	0.0.0.0	255.255.255.255	00:0b:82:01:fc:42	ff:ff:ff:ff:ff:ff
2	192.168.0.1	192.168.0.10	00:08:74:ab:f1:9b	00:0b:82:01:fc:42
3	0.0.0.0	255.255.255.255	00:0b:82:01:fc:42	ff:ff:ff:ff:ff:ff
4	192.168.0.1	192.168.0.10	00:08:74:ab:f1:9b	00:0b:82:01:fc:42

La raison de l'utilisation d'adresses unicast ou broadcast

Étape 1 : Discover

Le client ne connaît pas l'adresse du serveur ni sa propre IP.

→ Il envoie un ****Broadcast**** à tout le réseau.

+-----+

| Serveur DHCP |

| 192.168.1.1 |

+-----+

^

| Broadcast (IP=255.255.255.255, MAC=FF:FF:FF:FF:FF:FF)

| DHCP Discover

|

+-----+

| **Client DHCP** |

| **0.0.0.0** |

+-----+

Étape 2 : Offer

Le serveur propose une IP.

Si le client n'a pas encore d'IP, le message peut être en **Broadcast**, sinon parfois en **Unicast** si MAC connue.

+-----+

| Serveur DHCP |

| 192.168.1.1 |

+-----+

|

| Broadcast ou Unicast

| DHCP Offer

v

+-----+

| Client DHCP |

| 0.0.0.0 |

+-----+

Étape 3 : Request

Le client accepte l'offre et envoie un ****Broadcast**** pour informer tous les serveurs.

+-----+

| **Serveur DHCP** |

| **192.168.1.1** |

+-----+

^

| Broadcast

| DHCP Request

|

+-----+

| Client DHCP |

| 192.168.1.100 |

+-----+

Étape 4 : ACK

Le serveur confirme l'IP.

→ Ici, le message est généralement en ****Unicast**** car le client a maintenant une IP connue.

+-----+

| Serveur DHCP |

| 192.168.1.1 |

+-----+

|

| Unicast

| DHCP ACK

v

+-----+

| Client DHCP |

| 192.168.1.100 |

+-----+

Est-ce que le client avait une adresse IP préalable, qu'il souhaite éventuellement récupérer? Justifiez.

Oui, parfois.

Explication :

Si le client a déjà été connecté au réseau, il peut **avoir une IP précédemment attribuée par le serveur DHCP**.

Dans ce cas, lors du **DHCP Discover**, il peut demander **la même adresse IP qu'avant** (option "requested IP address" dans le paquet DHCP).

Le serveur peut :

- Lui attribuer la même IP si elle est encore disponible.
- Ou lui proposer une nouvelle IP si l'ancienne a été réassignée.

Ceci est particulièrement utile pour les **clients mobiles** (PC portables, imprimantes) qui se reconnectent souvent au même réseau.

Quelle adresse IP est attribuée au client ?

L'adresse IP attribuée provient d'un pool d'adresses géré par le serveur DHCP.

Exemple typique : 192.168.1.100 à 192.168.1.200.

Le serveur choisit une IP disponible et l'inclut dans le **DHCP Offer**.

Pour combien de temps ?

Le serveur DHCP attribue l'IP pour une **durée limitée appelée "lease time"**.

Exemple : 24 heures, 12 heures, ou même quelques minutes pour certains réseaux temporaires.

Une fois ce temps écoulé, le client doit :

- Renouveler son bail (lease) pour garder la même IP.
- Ou obtenir une nouvelle IP si le bail n'est pas renouvelé.

La durée du bail est indiquée dans le paquet DHCP **Offer** et **ACK**.

Est-ce que le client souhaite obtenir d'autres informations qu'une adresse IP ?

Oui, souvent.

Le DHCP ne fournit pas seulement l'adresse IP. Le client peut demander ou recevoir :

- **Masque de sous-réseau (Subnet Mask)**
 - **Passerelle par défaut (Default Gateway)**
 - **Serveurs DNS**
 - **Serveurs NTP** ou autres options spécifiques réseau
-

Outre l'adresse IP, quelles informations le serveur transmet-il au client ?

Le serveur DHCP peut transmettre plusieurs informations réseau importantes :

Type d'information	Exemple / Description
Masque de sous-réseau	Pour savoir quelle partie de l'IP est réseau et quelle partie est hôte
Passerelle par défaut	IP du routeur pour accéder à Internet
Serveurs DNS	Pour résoudre les noms de domaine en IP
Serveur WINS	(optionnel) pour certains réseaux Windows
Durée du bail (lease time)	Combien de temps l'IP est valide
Autres options DHCP	Serveur NTP, options spécifiques à l'entreprise, etc.

Ces informations permettent au client de **s'intégrer correctement dans le réseau** et de communiquer avec d'autres machines et Internet.

Explications supplémentaire

Étape 1 : DHCP Discover (Broadcast)

Client (0.0.0.0) ---> Broadcast (255.255.255.255)

Contenu :

1. MAC adresse du client
2. Option : IP souhaitée (si client avait une IP précédente)
3. Demande de configuration réseau (subnet, gateway, DNS)

Tout le réseau et les serveurs DHCP reçoivent ce message.

Étape 2 : DHCP Offer (Broadcast ou Unicast)

Serveur DHCP ---> Client (Broadcast si IP inconnue, Unicast si MAC connue)

Contenu :

1. Adresse IP proposée (ex: 192.168.1.100)
2. Masque de sous-réseau (ex: 255.255.255.0)
3. Passerelle par défaut (ex: 192.168.1.1)
4. Serveurs DNS (ex: 8.8.8.8)
5. Durée du bail (lease time, ex: 24h)
6. Autres options DHCP si nécessaires

Étape 3 : DHCP Request (Broadcast)

Client ---> Broadcast

Contenu :

1. Confirmation de l'adresse IP choisie (ex: 192.168.1.100)
2. MAC adresse du client
3. Demande des informations réseau complètes

Tous les serveurs DHCP savent maintenant quelle offre est acceptée.

Étape 4 : DHCP Acknowledge (Unicast)

Serveur DHCP ---> Client (Unicast)

Contenu :

1. Adresse IP attribuée (192.168.1.100)
2. Masque de sous-réseau
3. Passerelle par défaut
4. Serveurs DNS
5. Durée du bail
6. Autres options

Le client configure alors son interface réseau avec ces informations.

Résumé

Broadcast : utilisé quand le client ou le serveur n'est pas encore identifié.

Unicast : utilisé quand le destinataire est connu.

Informations envoyées : IP, masque, passerelle, DNS, durée du bail, MAC du client, options diverses.

3. DNS : Premières observations

Le DNS permet la résolution de noms de domaines en adresses IP. Il se base sur une organisation hiérarchique des noms de domaine pour répartir l'information sur un ensemble de serveurs DNS.

Dans ce TP, nous allons explorer les bases du DNS à travers l'observation de configuration DNS d'hôtes, d'observation d'échanges DNS avec Wireshark, et enfin, en expérimentant par nous-même des requêtes DNS via l'outil dig.

Certains de ces exercices sont inspirés du livre CNP3.

Démarrez Wireshark, puis créez du trafic réseau vers Internet, typiquement via votre navigateur Web. Filtrez pour afficher le trafic DNS. Pour les requêtes et les réponses affichées, analysez-les et essayez d'expliquer ce que vous observez, dont notamment :

Quels sont les champs de l'en-tête DNS ?

L'en-tête DNS (Domain Name System) est **fixe sur 12 octets** et contient les champs suivants :

Champ	Taille	Description
ID	16 bits	Identifiant de la requête pour faire correspondre réponse et requête.
QR	1 bit	Indique si le message est une requête (0) ou une réponse (1).
Opcode	4 bits	Type de requête : standard (0), inverse (1), etc.
AA	1 bit	“Authoritative Answer” : indique si la réponse provient d’un serveur faisant autorité.
TC	1 bit	“Truncated” : indique si le message est tronqué et nécessite TCP.
RD	1 bit	“Recursion Desired” : le client demande la récursion au serveur.
RA	1 bit	“Recursion Available” : le serveur supporte la récursion.
Z	3 bits	Réservé pour usage futur (toujours à 0).
RCODE	4 bits	Code retour : 0 = pas d’erreur, 3 = nom inexistant, etc.
QDCOUNT	16 bits	Nombre de questions dans la section Question.

ANCOUNT	16 bits	Nombre de réponses dans la section Answer.
NSCOUNT	16 bits	Nombre de serveurs de noms dans la section Authority.
ARCOUNT	16 bits	Nombre de records additionnels dans la section Additional.

Les sections **Question** et **Answer** suivent l'en-tête et contiennent les noms de domaine, types de requêtes (A, AAAA, MX, CNAME, etc.) et les réponses correspondantes.

Le DNS fonctionne-t-il au dessus de TCP ou d'UDP ?

Pouvez-vous expliquer pourquoi ?

Par défaut : UDP

- DNS utilise UDP pour la plupart des requêtes, car elles sont **courtes et rapides**.
- L'UDP est **sans connexion**, donc moins de surcharge et plus rapide pour de petites requêtes (A, AAAA, MX).

TCP

Utilisé si :

1. La réponse dépasse 512 octets (par exemple transfert de zones ou DNSSEC).
2. Requêtes de transfert de zone entre serveurs DNS (AXFR).

TCP garantit **la fiabilité et la fragmentation** pour les réponses longues.

Quel(s) numéro(s) de port sont utilisés par le DNS ?

Type de transport	Port
UDP	53
TCP	53

Le client envoie toujours sa requête vers le port 53 du serveur DNS.

La requête est envoyée à un **serveur DNS**, typiquement :

- Serveur DNS local de l'ISP ou configuré dans le système (ipconfig /all sous Windows).
- Pour les résolutions récursives, le serveur peut interroger d'autres serveurs DNS autoritaires.

La machine du client se situe généralement dans le **domaine de son réseau local** (LAN) mais elle interroge un serveur DNS pour des noms dans **n'importe quel domaine Internet**.

A qui est envoyée la requête DNS ?

Dans quel domaine se situe cette machine ?

The screenshot shows a Wireshark capture of DNS traffic on a Wi-Fi interface. The packet list at the top shows several DNS queries and responses. A red circle highlights packet 110, which is a standard query for 'images-ext-1.discordapp.net' sent to 192.168.1.1. A red arrow points from this packet to the packet details pane, which shows the query structure. A red squiggle is drawn over the packet details pane. The packet details pane shows the query for 'images-ext-1.discordapp.net'.

Voir dans quel domaine se situe la machine

Clique droit sur un paquet → **Follow** → **UDP Stream** pour suivre la conversation complète.

Dans la **requête DNS**, regarde le **nom de domaine demandé**.

Si tu veux connaître le **domaine du client** (ex : client.local ou example.com) :

- **Sur Windows** : **ipconfig /all** → Suffixe DNS principal montre le domaine du client.

- **Dans Wireshark** : les requêtes envoyées au serveur DNS contiennent parfois l'**option EDNS ou le nom du client** dans la section "Additional Records".
-

Observez les destinations de plusieurs requêtes DNS. Qu'en déduisez-vous ?

Observer les destinations de plusieurs requêtes DNS

Sélectionne tous les paquets DNS (filtre dns).

Clique sur **Statistics** → **Conversations** → **IPv4** ou **Endpoints** → **IPv4**.

- Cela te montre toutes les IP sources et destinations, avec le nombre de paquets échangés.

Tu verras les **destinations distinctes** des requêtes DNS :

- Serveur local de l'ISP
- Serveurs publics (Google DNS, Cloudflare, OpenDNS, etc.)
- Serveurs autoritaires finaux si la requête est récursive et que Wireshark capture le trafic entre ton client et ces serveurs.

Conclusion à tirer :

- Ton client n'envoie pas directement toutes les requêtes au serveur autoritaire final.
- Il envoie au serveur DNS configuré (ISP ou public), qui effectue la récursion.
- Les destinations observées dans Wireshark reflètent soit le serveur récursif, soit les étapes vers les serveurs autoritaires si capturés sur un réseau intermédiaire.

Astuces supplémentaires

- Pour **voir le nom de domaine demandé dans une colonne** : clic droit sur le champ Queries > Name → **Apply as Column**.
 - Pour **visualiser rapidement tous les serveurs contactés** : Statistics → Endpoints → IPv4, trie par "Destination".
 - Pour **voir les types de requêtes** : clique sur Queries > Type → ex : A, AAAA, MX, CNAME.
-

4. Résolveur DNS

A l'exercice précédent, vous avez observé que les échanges DNS s'effectuent avec un serveur DNS précis.

Comment votre machine sait-elle quel serveur DNS contacter ?

Lorsqu'un ordinateur veut résoudre un nom de domaine (ex : www.google.com), il a besoin d'un **serveur DNS**. La machine détermine ce serveur grâce à **sa configuration réseau**, qui peut

provenir de plusieurs sources :

Configuration automatique via DHCP

Sur la majorité des réseaux domestiques ou professionnels, le **serveur DHCP** attribue à la machine :

- Une **adresse IP**
- Un **masque de sous-réseau**
- Une **passerelle par défaut**
- Un ou plusieurs **serveurs DNS**

Le client DHCP reçoit alors l'information sur le serveur DNS et l'utilise automatiquement.

Configuration manuelle (statique)

- L'utilisateur ou l'administrateur peut **saisir directement l'adresse IP d'un serveur DNS** dans les paramètres réseau.
- Cela est utile pour utiliser des serveurs DNS publics (ex : Google 8.8.8.8, Cloudflare 1.1.1.1) ou pour des besoins internes (serveurs DNS locaux).

Pouvez-vous modifier cette configuration,

sur Windows ?

Et sur Linux ?

Modifier le serveur DNS sous Windows

Méthode 1 : Via l'interface graphique

1. Ouvre **Paramètres** → **Réseau et Internet** → **Modifier les options d'adaptateur**.
2. Clique droit sur la connexion utilisée → **Propriétés**.
3. Sélectionne **Protocole Internet version 4 (TCP/IPv4)** → **Propriétés**.
4. Coche **Utiliser l'adresse de serveur DNS suivante** et entre l'adresse désirée.
5. Valide et ferme les fenêtres.

Méthode 2 : Via la ligne de commande

Ouvre Invite de commandes en administrateur :

netsh interface ip set dns name="NomConnexion" static 8.8.8.8

Pour ajouter un DNS secondaire :

netsh interface ip add dns name="NomConnexion" 1.1.1.1 index=2

Vérifier la configuration :

ipconfig /all

Champ **Serveurs DNS** montre l'adresse actuelle.

Modifier le serveur DNS sous Linux

Méthode 1 : Modifier /etc/resolv.conf (temporaire)

Ouvrir le fichier avec un éditeur (exemple : nano) :

sudo nano /etc/resolv.conf

Ajouter ou modifier les lignes :

nameserver 8.8.8.8

nameserver 1.1.1.1

Sauvegarder et fermer.

Limite : ce fichier peut être **écrasé par DHCP** au redémarrage ou lors d'une reconnexion réseau.

Méthode 2 : Configurer via NetworkManager (permanent)

Interface graphique : Paramètres → Réseau → Connexion → Paramètres IPv4 → DNS manuel.

Ligne de commande :

nmcli con mod "NomConnexion" ipv4.dns "8.8.8.8 1.1.1.1"

nmcli con up "NomConnexion"

Méthode 3 : Pour les distributions utilisant systemd-resolved

Editer **/etc/systemd/resolved.conf** et définir :

DNS=8.8.8.8 1.1.1.1

FallbackDNS=1.1.1.1

Redémarrer le service :

sudo systemctl restart systemd-resolved

A votre avis, quand et comment votre machine a-t-elle reçu cette information ?

Quand et comment la machine reçoit l'information sur le serveur DNS ?

Quand ?

La machine reçoit l'information lors de la **configuration réseau**, typiquement au **démarrage** ou à la **connexion au réseau**.

Comment ?

1. **Via DHCP** : le serveur DHCP attribue : IP, passerelle, masque de sous-réseau et serveur(s) DNS.
 - La machine stocke cette information et l'utilise pour toutes les requêtes DNS.
2. **Manuellement** : si un serveur DNS est configuré statiquement, la machine utilise directement cette IP sans passer par DHCP.

Observation pratique :

- **Sur Windows** : **ipconfig /all** → champ **Serveurs DNS**
- **Sur Linux** : **cat /etc/resolv.conf** ou **systemd-resolve --status**

Sur Linux, observez les fichiers /etc/hosts, /etc/host.conf et /etc/resolv.conf. A quoi sert chaque fichier ?

Essayez d'expliquer chaque ligne observée.

Fichiers Linux liés à la résolution de noms

Fichier	Rôle	Contenu typique / lignes observées
/etc/hosts	Résolution locale statique des noms	Ex : 127.0.0.1 localhost → associe un nom à une IP locale sans interroger le DNS
/etc/host.conf	Paramètres de la résolution des noms (ordre et options)	Ex : order hosts,bind → indique à Linux de chercher d'abord dans /etc/hosts, puis via DNS (bind)
/etc/resolv.conf	Liste des serveurs DNS utilisés	Ex : nameserver 192.168.1.1 → IP du serveur DNS attribué par DHCP ou configuré manuellement

Explication ligne par ligne :

- **/etc/hosts** : chaque ligne = IP + nom(s) de machine. Prioritaire pour la résolution locale.
 - **/etc/host.conf** : contrôle l'ordre de recherche (hosts → local, bind → DNS) et options comme multi on pour plusieurs réponses.
 - **/etc/resolv.conf** : chaque ligne nameserver <IP> indique un serveur DNS à interroger dans l'ordre.
-

Sur Linux, modifiez la configuration DNS pour utiliser le serveur DNS public de Cloudflare, à l'IP 1.1.1.1. Vérifiez le changement en observant une nouvelle requête avec Wireshark. ATTENTION :

Cela ne fonctionne sans doute pas sur les PC Ephec, il est recommandé d'utiliser votre portable si vous en avez un. Le firewall de l'Ephec bloque le trafic DNS vers les résolveurs autres que le résolveur officiel. Ce n'est par contre pas le cas sur Eduroam.

Modifier le DNS sous Linux pour Cloudflare (1.1.1.1)

Méthode 1 : modification temporaire

sudo nano /etc/resolv.conf

Remplacer ou ajouter la ligne :

nameserver 1.1.1.1

Sauvegarder et fermer.

Limite : ce changement peut être écrasé si DHCP réécrit le fichier.

5. L'outil Dig

Le logiciel Dig permet de faire des requêtes DNS à un résolveur DNS. Dig est typiquement utilisé comme suit :

dig @<serveur> -t <type> <nom>

avec :

<serveur>, l'adresse IP ou le nom d'un serveur de nom ou d'un résolveur,

<type>, le type de Resource Record demandé (A pour une adresse IPv4, AAAA pour IPv6, MX pour le serveur mail,

<nom>, le nom de domaine pleinement qualifié (FQDN) de la machine dont on cherche l'IP.

Pour les exercices suivants, il est recommandé de capturer le trafic avec Wireshark afin d'observer les requêtes DNS échangées.

Sur une machine Linux, ouvrez un terminal, et essayez d'envoyer une requête DNS pour obtenir l'adresse IP (v4 et v6) des serveurs web

www.ephec.be,

www.facebook.com,

www.google.be,

www.whitehouse.gov,

ainsi que l'adresse IP du serveur mail de ephec.be.

Commandes Dig pour les exercices

IPv4	IPv6
dig www.ephec.be -t A	dig www.ephec.be -t AAAA
dig www.facebook.com -t A	dig www.facebook.com -t AAAA
dig www.google.be -t A	dig www.google.be -t AAAA
dig www.whitehouse.gov -t A	dig www.whitehouse.gov -t AAAA

Serveur mail (type MX)

dig ephec.be -t MX

Essayez d'expliquer chaque ligne de la réponse affichée par le logiciel.

Quel est le résolveur DNS qui vous fournit la réponse ?

Les informations sont-elles véhiculées par dessus TCP ou par dessus UDP ?

```
jeremie@pi-jeremie: ~  
Fichier  Édition  Onglets  Aide  
jeremie@pi-jeremie:~$ dig www.ephec.be -t A  
  
; <<>> DiG 9.18.33-1-deb12u2-Debian <<>> www.ephec.be -t A  
;; global options: +cmd  
;; Got answer:  
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 44434  
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1  
  
;; OPT PSEUDOSECTION:  
;; EDNS: version: 0, flags:; udp: 1232  
;; QUESTION SECTION:  
;www.ephec.be.                IN      A  
  
;; ANSWER SECTION:  
www.ephec.be.                30      IN      A      185.162.31.218  
  
;; Query time: 19 msec  
;; SERVER: 192.168.1.1#53(192.168.1.1) (UDP)  
;; WHEN: Sat Oct 11 09:18:22 CEST 2025  
;; MSG SIZE rcvd: 57  
  
jeremie@pi-jeremie:~$ ^C  
jeremie@pi-jeremie:~$ ^C  
jeremie@pi-jeremie:~$ S
```

Requête effectuée

dig www.ephec.be -t A

Type A → IPv4

Nom de domaine : **www.ephec.be**

Résolveur utilisé par défaut : celui configuré sur ta machine (**/etc/resolv.conf** ou **systemd-resolved**)

Analyse ligne par ligne de la réponse

;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 44434

opcode: QUERY → requête standard DNS

status: NOERROR → pas d'erreur, la requête a été traitée correctement

id: 44434 → identifiant unique pour cette requête/réponse

:: flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

qr = réponse (query response)

rd = récursion demandée par le client

ra = récursion disponible sur le serveur

QUERY:1 → 1 question posée

ANSWER:1 → 1 réponse reçue

AUTHORITY:0 → pas de section autoritaire dans cette réponse

ADDITIONAL:1 → section additionnelle (OPT/EDNS)

:: OPT PSEUDOSECTION:

; EDNS: version: 0, flags;; udp: 1232

EDNS → Extension DNS pour paquets >512 octets

Taille maximale UDP = 1232 octets

:: QUESTION SECTION:

;www.ephec.be. IN A

Nom demandé : www.ephec.be

Type : A → IPv4

:: ANSWER SECTION:

www.ephec.be. 30 IN A 185.162.31.218

Résultat : l'adresse IPv4 est 185.162.31.218

TTL = 30 secondes (durée de validité dans le cache)

:: Query time: 19 msec

:: SERVER: 192.168.1.1#53(UDP)

:: WHEN: Sat Oct 11 09:18:22 CEST 2025

:: MSG SIZE rcvd: 57

Temps de réponse : 19 ms

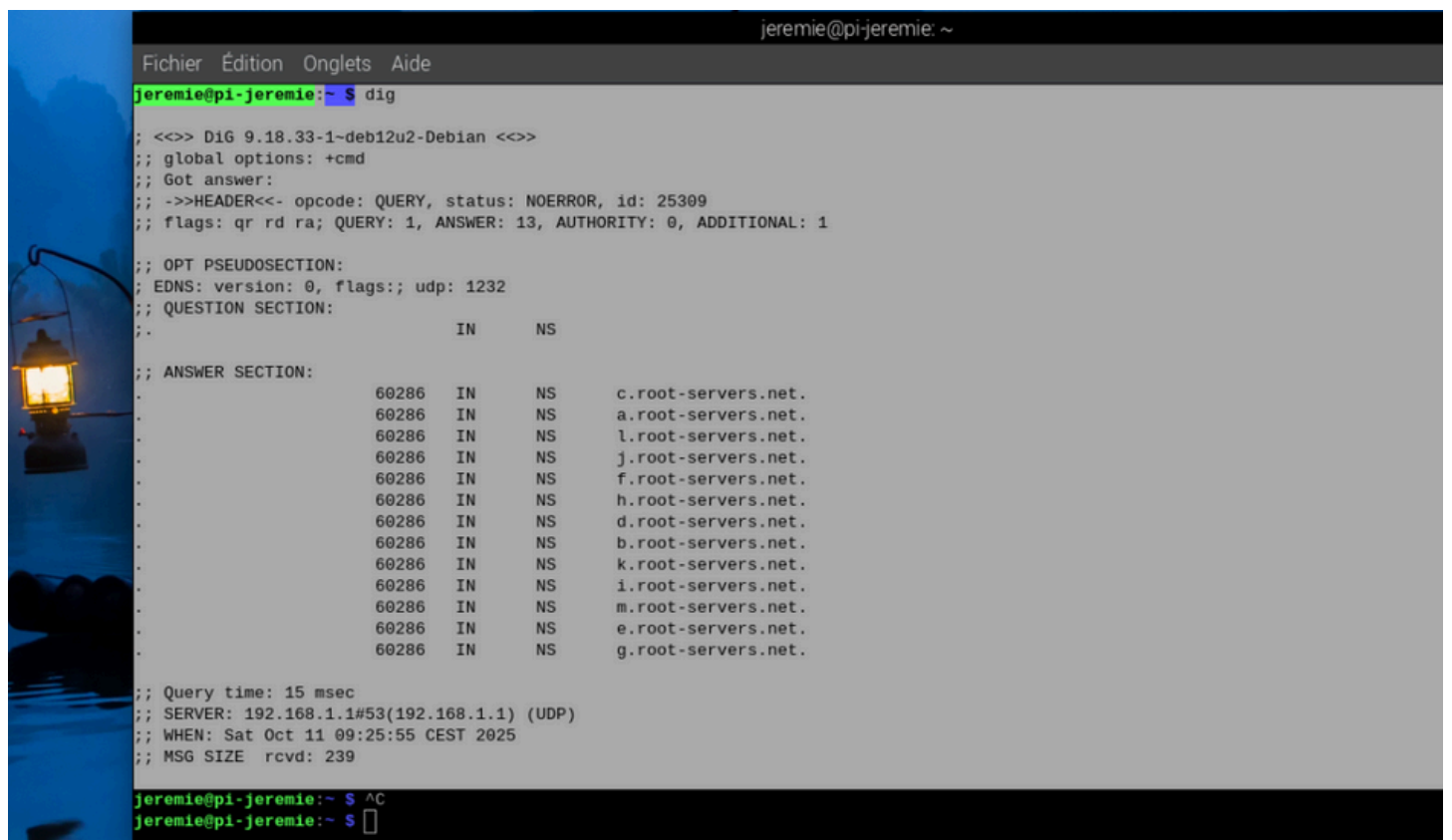
Serveur interrogé : 192.168.1.1 (probablement ton routeur/serveur DHCP local)

Protocole : UDP port 53

Résumé des informations demandées

Question	Réponse
Quel est le résolveur DNS ?	192.168.1.1 (configuré sur la machine via DHCP ou manuellement)
Protocole utilisé ?	UDP (port 53)
Quelle IP est retournée ?	185.162.31.218
TTL ?	30 secondes
Informations supplémentaires reçues ?	EDNS info pour paquets plus grands (taille UDP = 1232)

Quelle réponse obtenez-vous lorsque vous invoquez Dig sans nom de domaine en paramètre ?



```
jeremie@pi-jeremie: ~
Fichier Édition Onglets Aide
jeremie@pi-jeremie:~$ dig

; <<>> DiG 9.18.33-1~deb12u2-Debian <<>>
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 25309
;; flags: qr rd ra; QUERY: 1, ANSWER: 13, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
; .                               IN      NS

;; ANSWER SECTION:
60286 IN      NS      c.root-servers.net.
60286 IN      NS      a.root-servers.net.
60286 IN      NS      l.root-servers.net.
60286 IN      NS      j.root-servers.net.
60286 IN      NS      f.root-servers.net.
60286 IN      NS      h.root-servers.net.
60286 IN      NS      d.root-servers.net.
60286 IN      NS      b.root-servers.net.
60286 IN      NS      k.root-servers.net.
60286 IN      NS      i.root-servers.net.
60286 IN      NS      m.root-servers.net.
60286 IN      NS      e.root-servers.net.
60286 IN      NS      g.root-servers.net.

;; Query time: 15 msec
;; SERVER: 192.168.1.1#53(192.168.1.1) (UDP)
;; WHEN: Sat Oct 11 09:25:55 CEST 2025
;; MSG SIZE rcvd: 239

jeremie@pi-jeremie:~$ ^C
jeremie@pi-jeremie:~$
```

Analyse de la commande dig seule

Header et informations globales

;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 25309

;; flags: qr rd ra; QUERY: 1, ANSWER: 13, AUTHORITY: 0, ADDITIONAL: 1

opcode: QUERY → requête standard

status: NOERROR → pas d'erreur

QUERY: 1 → 1 question (ici spéciale, pas de nom de domaine)

ANSWER: 13 → 13 réponses (serveurs racines)

AUTHORITY: 0 → pas de section autoritaire

ADDITIONAL: 1 → section additionnelle pour EDNS

Question (Question Section)

:: QUESTION SECTION:

;; IN NS

La requête porte sur le **domaine racine** .

Type : NS → Name Server

Cette requête demande donc la **liste des serveurs racines** du DNS.

Réponse (Answer Section)

:: ANSWER SECTION:

. 60286 IN NS c.root-servers.net.

. 60286 IN NS/a.root-servers.net.

...

Liste des **13 serveurs racines** (a.root-servers.net à m.root-servers.net)

TTL = 60286 secondes → temps pendant lequel la réponse peut être mise en cache

Cette réponse montre **tous les serveurs racines connus et accessibles** depuis ton résolveur.

Informations complémentaires

:: Query time: 15 msec

:: SERVER: 192.168.1.1#53(UDP)

:: WHEN: Sat Oct 11 09:25:55 CEST 2025

:: MSG SIZE rcvd: 239

Temps de réponse : 15 ms → rapide car le résolveur peut avoir mis en cache les serveurs racines

Serveur utilisé : 192.168.1.1 (ton routeur / serveur DHCP)

Protocole : UDP port 53

Analyse et explication

Que fait dig sans nom de domaine ?

Elle interroge le résolveur pour obtenir la liste des serveurs **racines DNS**.

Utile pour comprendre comment les requêtes DNS peuvent être résolues depuis la racine.

Résolveur utilisé

Toujours 192.168.1.1 → ton serveur local (routeur / DHCP).

Protocole

UDP port 53, comme pour les requêtes classiques.

Informations véhiculées

La réponse contient les **13 serveurs racines**, qui permettent ensuite de résoudre n'importe quel domaine récursivement.

Note : si tu ne précises pas **@<serveur>**, dig interrogera **le résolveur DNS configuré sur ta machine** (via **/etc/resolv.conf** ou **systemd-resolved**).

Par défaut, dig interroge le résolveur configuré sur la machine. Vous pouvez également interroger n'importe quel serveur de noms directement (le mode d'emploi de dig est disponible en tapant man dig dans la ligne de commande).

Interrogez récursivement les serveurs de noms nécessaires pour obtenir l'adresse IP de l'hôte www.ephec.be. Indiquez à chaque fois, pour chaque domaine et sous-domaine, le nom du serveur DNS qui a autorité.

Étape 1 : Identifier les serveurs racines

La racine DNS (.) est autoritaire pour le sommet de la hiérarchie. On peut interroger directement un serveur racine avec Dig :

dig @a.root-servers.net www.ephec.be

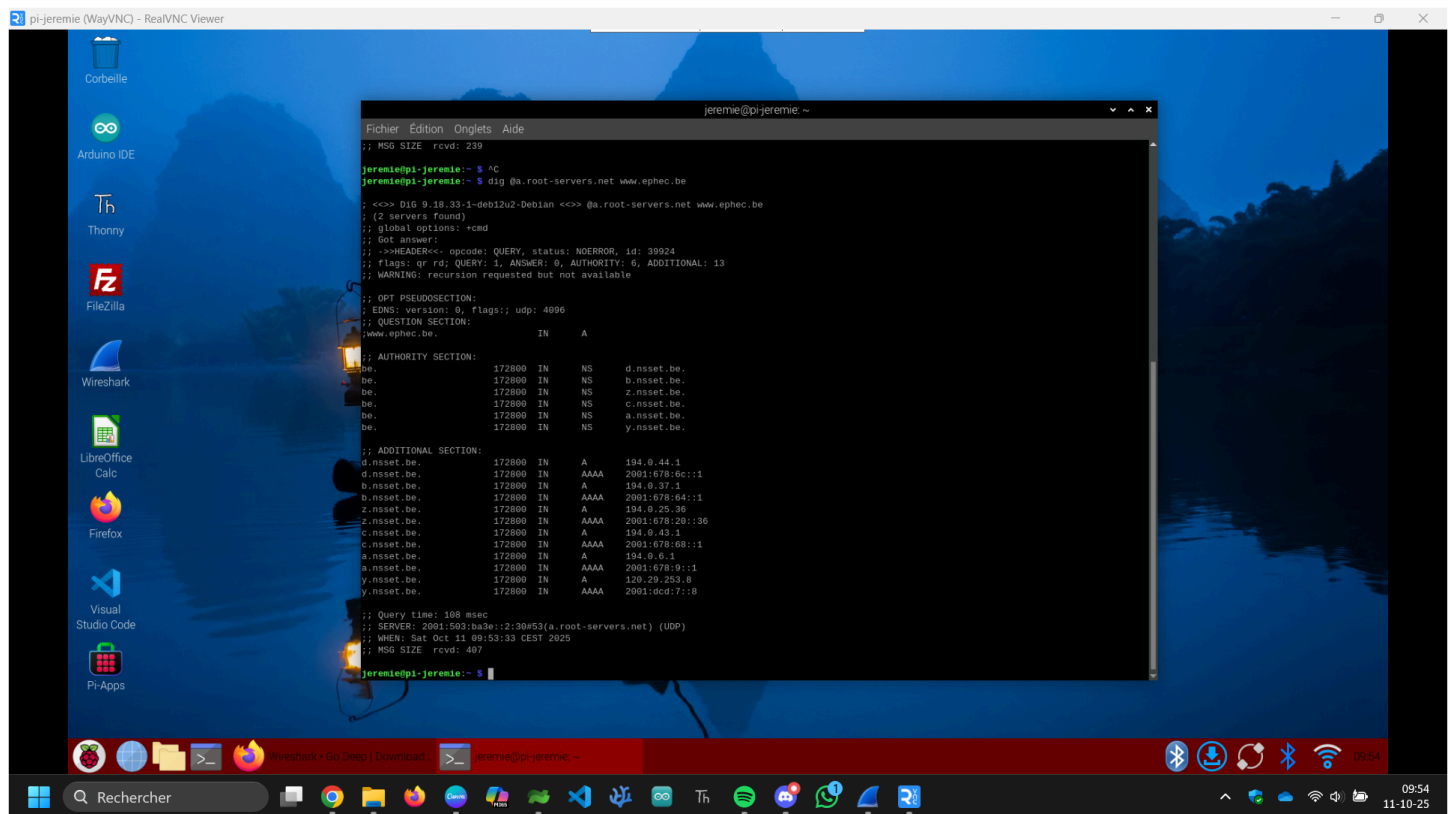
Résultat attendu :

Le serveur racine ne connaît pas directement l'IP de www.ephec.be.

Il retourne une **liste de serveurs TLD pour .be** (ex : a.nic.be, b.nic.be, ...).

Autorité : Serveur racine (ex : a.root-servers.net)

Domaine concerné : . (root)



```
jeremie@pi-jeremie:~$ ^C
jeremie@pi-jeremie:~$ dig @a.root-servers.net www.ephec.be

;; <>> DIG 9.18.33-1-debian <>> @a.root-servers.net www.ephec.be
(2 servers found)
;; global options: +cmd
;; Got answer:
;;->>HEADER<<: opcode: QUERY, status: NOERROR, id: 39924
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 6, ADDITIONAL: 13
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.ephec.be.                IN      A
;; AUTHORITY SECTION:
be. 172800 IN NS d.nsset.be.
be. 172800 IN NS b.nsset.be.
be. 172800 IN NS z.nsset.be.
be. 172800 IN NS c.nsset.be.
be. 172800 IN NS a.nsset.be.
be. 172800 IN NS y.nsset.be.
;; ADDITIONAL SECTION:
d.nsset.be. 172800 IN A 194.0.44.1
d.nsset.be. 172800 IN AAAA 2001:678:6c::1
b.nsset.be. 172800 IN A 194.0.37.1
b.nsset.be. 172800 IN AAAA 2001:678:64::1
z.nsset.be. 172800 IN A 194.0.26.36
z.nsset.be. 172800 IN AAAA 2001:678:20::36
c.nsset.be. 172800 IN A 194.0.43.1
c.nsset.be. 172800 IN AAAA 2001:678:68::1
a.nsset.be. 172800 IN A 194.0.6.1
a.nsset.be. 172800 IN AAAA 2001:678:9::1
y.nsset.be. 172800 IN A 126.26.253.8
y.nsset.be. 172800 IN AAAA 2001:6cd:7::8

;; Query time: 198 msec
;; SERVER: 2001:503:ba3e:12:30::53(a.root-servers.net) (UDP)
;; WHEN: Sat Oct 11 09:53:33 CEST 2025
;; MSG SIZE rcvd: 497

jeremie@pi-jeremie:~$
```

Première requête : serveur racine

dig @a.root-servers.net www.ephec.be

Résultat clé :

;; AUTHORITY SECTION:

be. 172800 IN NS d.nsset.be.

be. 172800 IN NS b.nsset.be.

be. 172800 IN NS z.nsset.be.

...

;; ADDITIONAL SECTION:

d.nsset.be. 172800 IN A 194.0.44.1

...

Le serveur racine **ne connaît pas directement l'IP de www.ephec.be**

Il fournit **les serveurs autoritaires pour le domaine .be** (d.nsset.be, b.nsset.be, etc.)

L'**Additional section** contient les adresses IP de ces serveurs (A pour IPv4, AAAA pour IPv6) → c'est ce qu'il faut utiliser pour les requêtes suivantes.

Étape 2 : Interroger le serveur TLD pour .be

dig @a.nic.be www.ephec.be

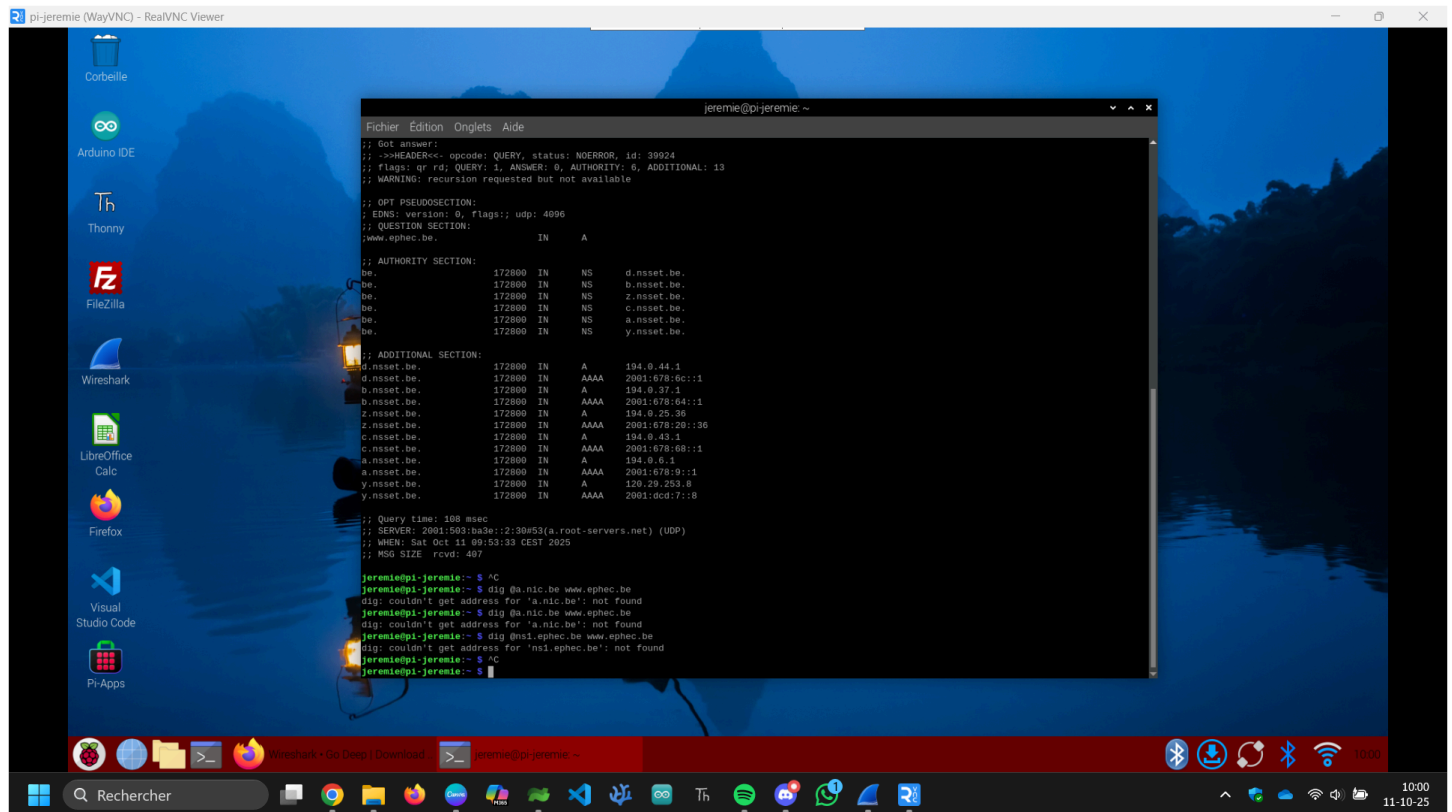
Résultat attendu :

Le serveur TLD .be ne connaît pas directement l'IP de www.ephec.be.

Il retourne le ou les **serveurs autoritaires pour le domaine ephec.be** (ex : ns1.ephec.be)

Autorité : Serveur TLD .be (ex : a.nic.be)

Domaine concerné : .be



```
pi-jeremie (WayVNC) - RealVNC Viewer
jeremie@pi-jeremie: ~
Fichier Édition Onglets Aide
;; Got answer:
;; >>>HEADER<<< opcode: QUERY, status: NOERROR, id: 39924
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 6, ADDITIONAL: 13
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: udp: 4096
;; QUESTION SECTION:
;www.ephec.be.                IN      A

;; AUTHORITY SECTION:
be.                172800 IN      NS      d.nset.be.
be.                172800 IN      NS      b.nset.be.
be.                172800 IN      NS      z.nset.be.
be.                172800 IN      NS      c.nset.be.
be.                172800 IN      NS      a.nset.be.
be.                172800 IN      NS      y.nset.be.

;; ADDITIONAL SECTION:
d.nset.be.         172800 IN      A      194.0.44.1
d.nset.be.         172800 IN      AAAA   2001:678:6c::1
b.nset.be.         172800 IN      A      194.0.37.1
b.nset.be.         172800 IN      AAAA   2001:678:64::1
z.nset.be.         172800 IN      A      194.0.25.36
z.nset.be.         172800 IN      AAAA   2001:678:20::36
c.nset.be.         172800 IN      A      194.0.43.1
c.nset.be.         172800 IN      AAAA   2001:678:68::1
a.nset.be.         172800 IN      A      194.0.6.1
a.nset.be.         172800 IN      AAAA   2001:678:9::1
y.nset.be.         172800 IN      A      120.29.253.8
y.nset.be.         172800 IN      AAAA   2001:dcd:7::8

;; Query time: 108 msec
;; SERVER: 2001:503:ba3e::12:30#53(a.root-servers.net) (UDP)
;; WHEN: Sat Oct 11 09:53:33 CEST 2025
;; MSG SIZE  rcvd: 497

jeremie@pi-jeremie:~$ ^C
jeremie@pi-jeremie:~$ dig @a.nic.be www.ephec.be
dig: couldn't get address for 'a.nic.be': not found
jeremie@pi-jeremie:~$ dig @a.nic.be www.ephec.be
dig: couldn't get address for 'a.nic.be': not found
jeremie@pi-jeremie:~$ dig @ns1.ephec.be www.ephec.be
dig: couldn't get address for 'ns1.ephec.be': not found
jeremie@pi-jeremie:~$ ^C
jeremie@pi-jeremie:~$
```

Pourquoi dig @a.nic.be ne fonctionne pas

dig @a.nic.be www.ephec.be

Erreur : couldn't get address for 'a.nic.be': not found

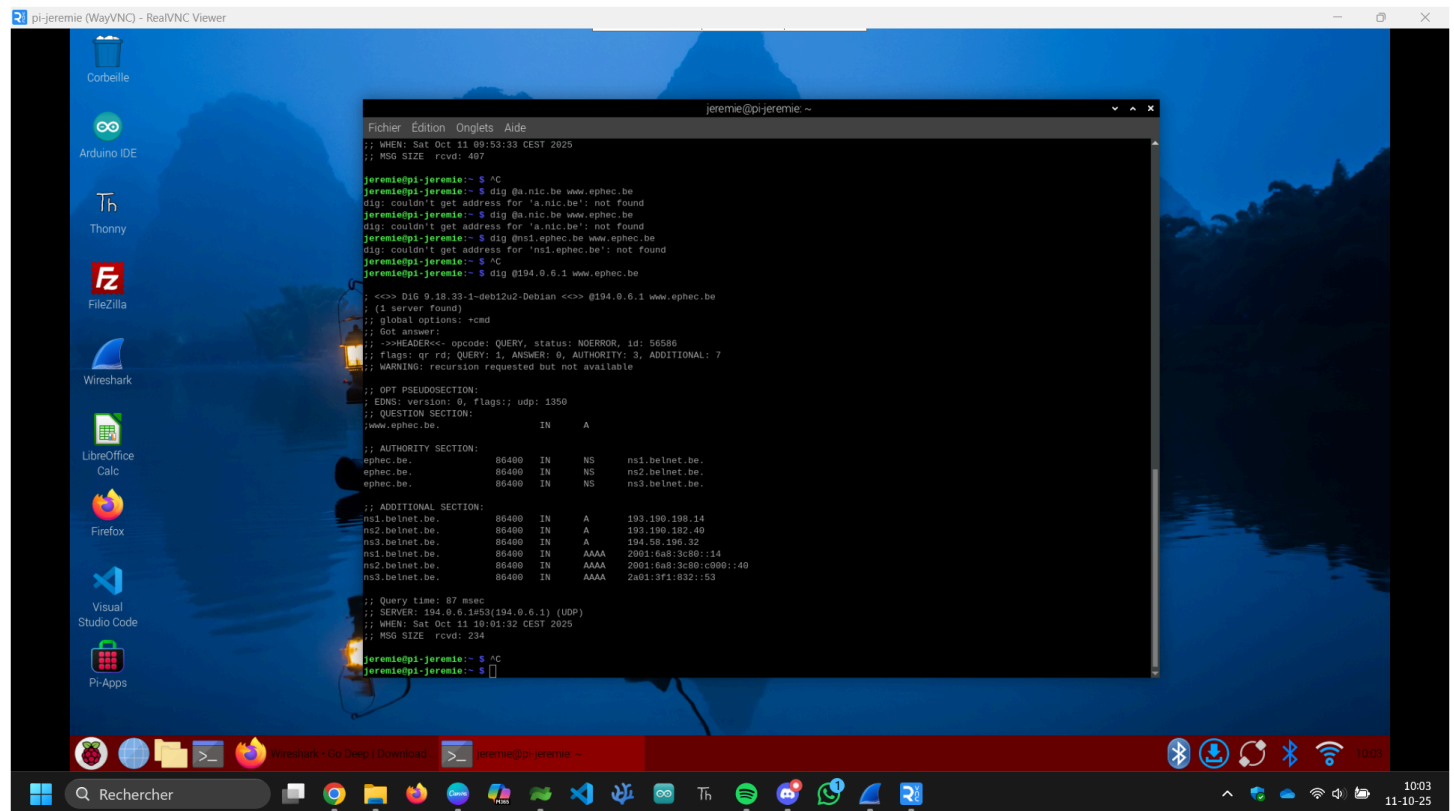
Problème : ton système **ne peut pas résoudre le nom a.nic.be** car il n'a pas encore accès à un serveur DNS pour résoudre ce nom.

Solution : utiliser l'**IP directement**, trouvée dans la section ADDITIONAL précédente.

Exemple :

dig @194.0.6.1 www.ephec.be

Ici, **194.0.6.1** correspond à **a.nsset.be** ou autre serveur autoritaire du TLD .be (tu choisis l'IP que tu veux interroger).



```
Fichier  Edition  Onglets  Aide
jeremie@pi-jeremie: ~
;; WHEN: Sat Oct 11 09:53:33 CEST 2025
;; MSG SIZE rcvd: 407

jeremie@pi-jeremie:~$ AC
jeremie@pi-jeremie:~$ dig @a.nic.be www.ephec.be
dig: couldn't get address for 'a.nic.be': not found
jeremie@pi-jeremie:~$ dig @a.nic.be www.ephec.be
dig: couldn't get address for 'a.nic.be': not found
jeremie@pi-jeremie:~$ dig @ns1.ephec.be www.ephec.be
dig: couldn't get address for 'ns1.ephec.be': not found
jeremie@pi-jeremie:~$ AC
jeremie@pi-jeremie:~$ dig @194.0.6.1 www.ephec.be

<<>> DIG 9.18.33-1-deb12u2-Debian <<>> @194.0.6.1 www.ephec.be
(1 server found)
;; global options: +cmd
;; Got answer:
;;->HEADER<<: opcode: QUERY, status: NOERROR, id: 56586
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 3, ADDITIONAL: 7
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags: udp: 1350
;; QUESTION SECTION:
;; www.ephec.be.                IN      A

;; AUTHORITY SECTION:
ephec.be.      86400 IN  NS   ns1.belnet.be.
ephec.be.      86400 IN  NS   ns2.belnet.be.
ephec.be.      86400 IN  NS   ns3.belnet.be.

;; ADDITIONAL SECTION:
ns1.belnet.be. 86400 IN  A    193.190.198.14
ns2.belnet.be. 86400 IN  A    193.190.192.40
ns3.belnet.be. 86400 IN  A    194.58.196.32
ns1.belnet.be. 86400 IN  AAAA  2001:6a8:3c80:14
ns2.belnet.be. 86400 IN  AAAA  2001:6a8:3c80:c000:40
ns3.belnet.be. 86400 IN  AAAA  2a01:3f1:832::53

;; Query time: 87 msec
;; SERVER: 194.0.6.1#53(194.0.6.1) (UDP)
;; WHEN: Sat Oct 11 10:01:32 CEST 2025
;; MSG SIZE rcvd: 234

jeremie@pi-jeremie:~$ AC
jeremie@pi-jeremie:~$
```

Serveur interrogé

dig @194.0.6.1 www.ephec.be

194.0.6.1 → IP du serveur TLD .be obtenu à l'étape précédente depuis le serveur racine

Protocole : UDP, port 53

Analyse de la réponse

:: AUTHORITY SECTION:

ephec.be. 86400 IN NS ns1.belnet.be.

ephec.be. 86400 IN NS ns2.belnet.be.

ephec.be. 86400 IN NS ns3.belnet.be.

Le serveur TLD .be **ne connaît pas directement l'IP de www.ephec.be**

Il renvoie **les serveurs autoritaires pour le domaine ephec.be** :

- **ns1.belnet.be.**
- **ns2.belnet.be.**
- **ns3.belnet.be.**

TTL = 86400 sec (24 h) → durée pendant laquelle ces infos peuvent être mises en cache

:: ADDITIONAL SECTION:

ns1.belnet.be. 86400 IN A 193.190.198.14

ns2.belnet.be. 86400 IN A 193.190.182.40

ns3.belnet.be. 86400 IN A 194.58.196.32

ns1.belnet.be. 86400 IN AAAA 2001:6a8:3c80::14

ns2.belnet.be. 86400 IN AAAA 2001:6a8:3c80:c000::40

ns3.belnet.be. 86400 IN AAAA 2a01:3f1:832::53

La section **Additional** contient les **adresses IP de ces serveurs autoritaires**, pour que le client puisse les interroger directement.

On voit ici à la fois **IPv4 (A)** et **IPv6 (AAAA)**

Interprétation

Domaine demandé : **www.ephec.be**

Serveur autoritaire pour la zone **.be** : **194.0.6.1 (TLD)**

Serveurs autoritaires pour **ephec.be** : **ns1.belnet.be, ns2.belnet.be, ns3.belnet.be**

La machine n'a pas encore l'IP finale, mais a maintenant toutes les informations nécessaires pour la récupérer dans l'étape suivante.

Étape 3 : Interroger le serveur autoritaire pour ephec.be

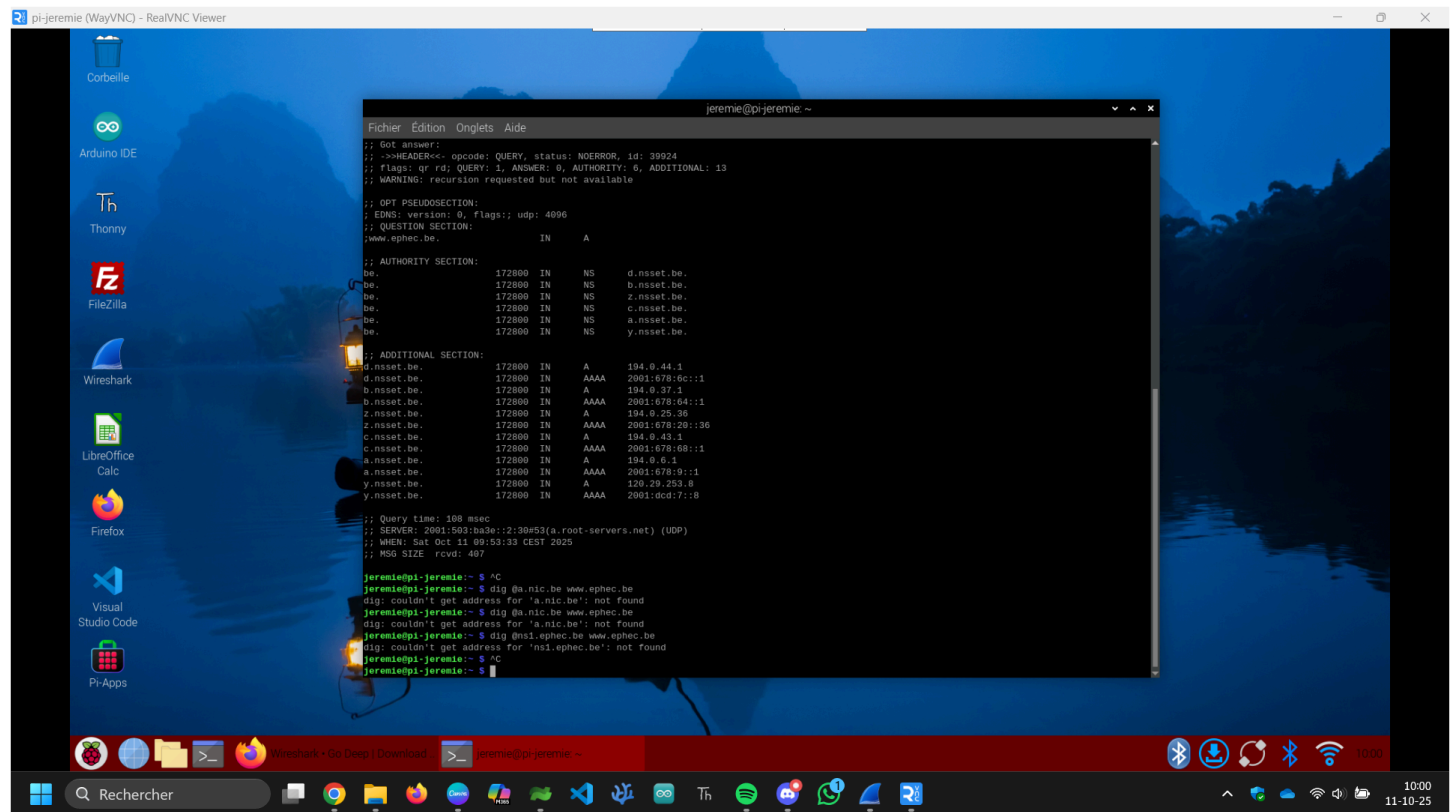
dig @ns1.ephec.be www.ephec.be

Résultat attendu :

Le serveur autoritaire connaît l'IP exacte de **www.ephec.be** : **185.162.31.218**

Autorité : Serveur autoritaire pour **ephec.be (ns1.ephec.be)**

Domaine concerné : **ephec.be**



```
jeremie@pi-jeremie: ~  
Fichier Édition Onglets Aide  
;; Got answer:  
;; -->HEADER: opcode: QUERY, status: NOERROR, id: 39924  
;; flags: qr rd, QUERY: 1, ANSWER: 0, AUTHORITY: 6, ADDITIONAL: 13  
;; WARNING: recursion requested but not available  
  
;; OPT PSEUDOSECTION:  
; EDNS: version: 0, flags: udp: 4096  
;; QUESTION SECTION:  
; www.ephec.be. IN A  
  
;; AUTHORITY SECTION:  
be. 172800 IN NS d.nsset.be.  
be. 172800 IN NS b.nsset.be.  
be. 172800 IN NS z.nsset.be.  
be. 172800 IN NS c.nsset.be.  
be. 172800 IN NS a.nsset.be.  
be. 172800 IN NS y.nsset.be.  
  
;; ADDITIONAL SECTION:  
d.nsset.be. 172800 IN A 194.0.44.1  
d.nsset.be. 172800 IN AAAA 2001:678:ac::1  
b.nsset.be. 172800 IN A 194.0.37.1  
b.nsset.be. 172800 IN AAAA 2001:678:64::1  
z.nsset.be. 172800 IN A 194.0.25.80  
z.nsset.be. 172800 IN AAAA 2001:678:20::30  
c.nsset.be. 172800 IN A 194.0.43.1  
c.nsset.be. 172800 IN AAAA 2001:678:68::1  
a.nsset.be. 172800 IN A 194.0.6.1  
a.nsset.be. 172800 IN AAAA 2001:678:9::1  
y.nsset.be. 172800 IN A 120.29.253.8  
y.nsset.be. 172800 IN AAAA 2001:dc0:7::8  
  
;; Query time: 188 msec  
;; SERVER: 2001:503:ba3e::2:30:53(a.root-servers.net) (UDP)  
;; WHEN: Sat Oct 11 09:53:33 CEST 2025  
;; MSG SIZE rcvd: 407  
  
jeremie@pi-jeremie:~$ PC  
jeremie@pi-jeremie:~$ dig @a.nic.be www.ephec.be  
dig: couldn't get address for 'a.nic.be': not found  
jeremie@pi-jeremie:~$ dig @a.nic.be www.ephec.be  
dig: couldn't get address for 'a.nic.be': not found  
jeremie@pi-jeremie:~$ dig @ns1.ephec.be www.ephec.be  
dig: couldn't get address for 'ns1.ephec.be': not found  
jeremie@pi-jeremie:~$ PC  
jeremie@pi-jeremie:~$
```

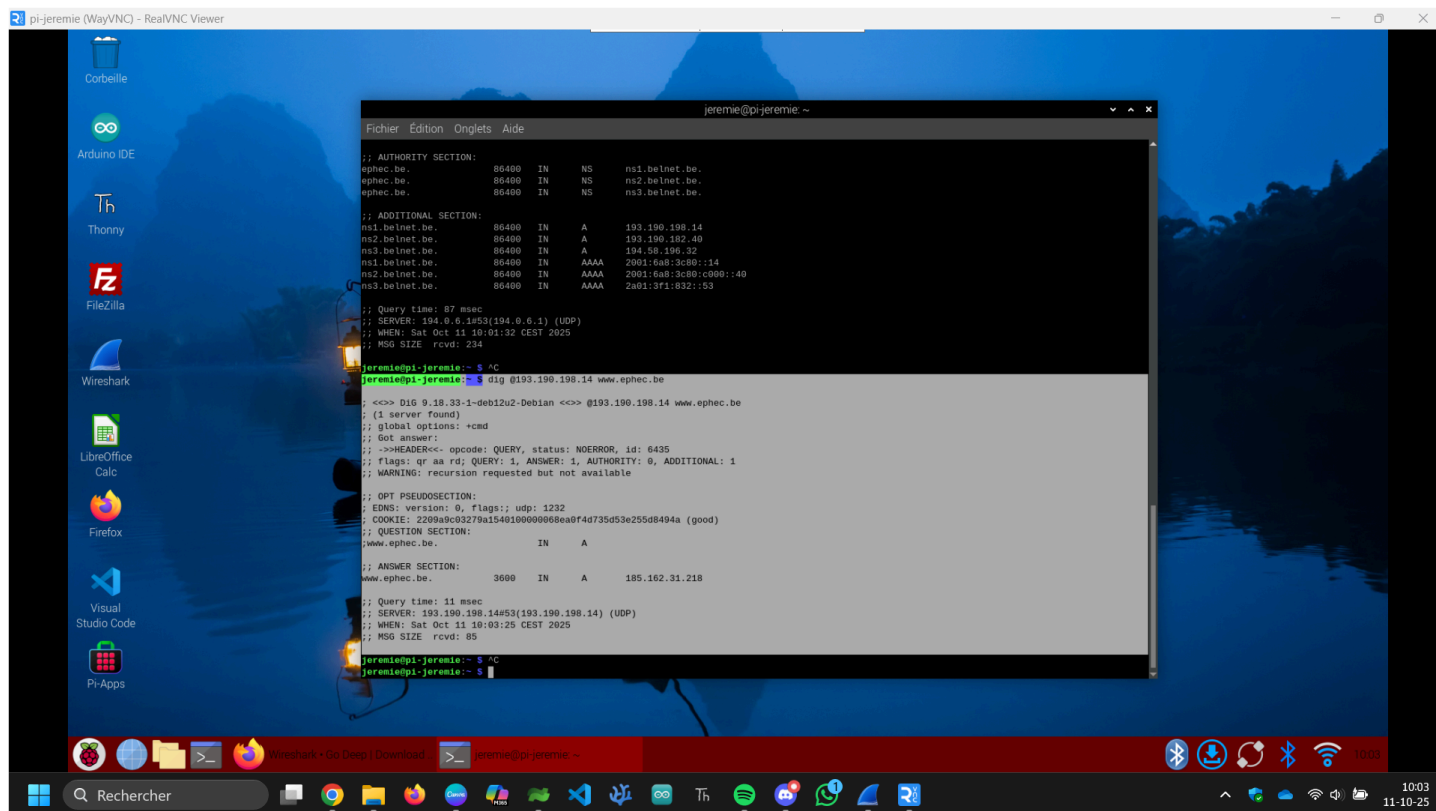
Même problème pour ns1.ephec.be

dig @ns1.ephec.be www.ephec.be

Erreur : couldn't get address for 'ns1.ephec.be': not found

Problème similaire : la machine ne connaît pas l'IP de ns1.ephec.be.

Solution : consulter les enregistrements NS pour ephec.be (via le TLD .be) et utiliser directement l'IP du serveur autoritaire.



Serveur interrogé

dig @193.190.198.14 www.ephec.be

193.190.198.14 → IP de ns1.belnet.be, serveur autoritaire pour ephec.be

Protocole utilisé : UDP, port 53

Recursion : désactivée (recursion requested but not available) → normal pour un serveur autoritaire

Section clé : ANSWER

:: ANSWER SECTION:

www.ephec.be. 3600 IN A 185.162.31.218

L'adresse IPv4 finale de www.ephec.be est **185.162.31.218**

TTL = 3600 sec → cette réponse peut être mise en cache par ton résolveur local pendant 1 heure

Section AUTHORITY

:: AUTHORITY: 0

Pas de serveur supplémentaire mentionné ici, car on a déjà atteint le serveur **autoritaire final** pour ce domaine

Chemin complet de résolution récursive

Étape	Domaine demandé	Serveur interrogé	Serveur autoritaire renvoyé	Résultat
1	www.ephec.be	a.root-servers.net	TLD .be (d.nsset.be, etc.)	IP du TLD
2	www.ephec.be	194.0.6.1 (TLD .be)	Serveurs autoritaires ephec.be (ns1.belnet.be, ns2.belnet.be)	IP des serveurs autoritaires
3	www.ephec.be	193.190.198.14 (ns1.belnet.be)	–	IP finale : 185.162.31.218

Résultat final : www.ephec.be = **185.162.31.218**

Observations importantes

Chaque serveur connaît seulement la zone qu'il administre :

- Racine → TLD
- TLD → Serveur autoritaire du domaine
- Serveur autoritaire → IP finale

Dig par défaut utilise ton **résolveur local**, mais ici tu as fait la résolution **pas à pas** avec les IP directes.

Protocole : UDP, port 53, sauf si la réponse est trop grande ou si tu forces TCP (dig +tcp).

TTL : permet le caching pour réduire les requêtes répétitives.

Ce processus illustre comment la **résolution DNS récursive** fonctionne :

Client → Résolveur local → Serveurs racines → Serveur TLD → Serveur autoritaire → IP finale

Remarques importantes

On peut répéter ce processus avec n'importe quel domaine ou sous-domaine.

Le client final (ta machine) interroge **toujours son résolveur local**.

Les serveurs racines et TLD ne fournissent **jamais directement l'adresse finale** si le domaine n'est pas à leur charge.

Le protocole DNS peut être utilisé au dessus d'UDP ou de TCP. Beaucoup de serveurs DNS préfèrent utiliser UDP car il consomme moins de ressources sur le serveur. Cependant, TCP est utile lorsqu'une réponse de taille conséquente est attendue.

Vous pouvez forcer dig à utiliser tcp en utilisant dig +tcp.

Utilisez UDP puis TCP pour faire une requête à un des serveurs racine.

Quel protocole est le plus rapide ? (Conseil : Utilisez la commande time)

Préparation

Tu peux mesurer le temps d'exécution de dig avec la commande time :

time dig @a.root-servers.net www.ephec.be

Par défaut, Dig utilise **UDP**.

Pour forcer TCP :

time dig +tcp @a.root-servers.net www.ephec.be


```

jeremie@pi-jeremie:~$ dig
jeremie@pi-jeremie:~$ time dig @a.root-servers.net www.ephec.be
;; <>> DIG 9.18.33-1-deb12u2-Debian <>> @a.root-servers.net www.ephec.be
;; (2 servers found)
;; global options: +cmd
;; Got answer:
;; -->HEADER<>< opcode: QUERY, status: NOERROR, id: 65497
;; flags: qr rd; QUERY: 1, ANSWER: 6, AUTHORITY: 6, ADDITIONAL: 13
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags: udp: 4096
;; QUESTION SECTION:
;www.ephec.be.                IN      A

;; AUTHORITY SECTION:
be. 172800 IN NS d.nsset.be.
be. 172800 IN NS b.nsset.be.
be. 172800 IN NS z.nsset.be.
be. 172800 IN NS c.nsset.be.
be. 172800 IN NS a.nsset.be.
be. 172800 IN NS y.nsset.be.

;; ADDITIONAL SECTION:
d.nsset.be. 172800 IN A 194.0.44.1
d.nsset.be. 172800 IN AAAA 2001:678:6c::1
b.nsset.be. 172800 IN A 194.0.37.1
b.nsset.be. 172800 IN AAAA 2001:678:64::1
z.nsset.be. 172800 IN A 194.0.25.36
z.nsset.be. 172800 IN AAAA 2001:678:20::36
c.nsset.be. 172800 IN A 194.0.43.1
c.nsset.be. 172800 IN AAAA 2001:678:68::1
a.nsset.be. 172800 IN A 194.0.6.1
a.nsset.be. 172800 IN AAAA 2001:678:9::1
y.nsset.be. 172800 IN A 129.29.253.8
y.nsset.be. 172800 IN AAAA 2001:dcd:7::8

;; Query time: 111 msec
;; SERVER: 2001:503:b3e::2:30e53(a.root-servers.net) (UDP)
;; WHEN: Sat Oct 11 19:06:56 CEST 2025
;; MSG SIZE rcvd: 407

real 0m0.154s
user 0m0.000s
sys 0m0.004s
jeremie@pi-jeremie:~$

jeremie@pi-jeremie:~$ time dig +tcp @a.root-servers.net www.ephec.be
;; <>> DIG 9.18.33-1-deb12u2-Debian <>> +tcp @a.root-servers.net www.ephec.be
;; (2 servers found)
;; global options: +cmd
;; Got answer:
;; -->HEADER<>< opcode: QUERY, status: NOERROR, id: 65498
;; flags: qr rd; QUERY: 1, ANSWER: 6, AUTHORITY: 6, ADDITIONAL: 13
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags: udp: 4096
;; QUESTION SECTION:
;www.ephec.be.                IN      A

;; AUTHORITY SECTION:
be. 172800 IN NS d.nsset.be.
be. 172800 IN NS b.nsset.be.
be. 172800 IN NS z.nsset.be.
be. 172800 IN NS c.nsset.be.
be. 172800 IN NS a.nsset.be.
be. 172800 IN NS y.nsset.be.

;; ADDITIONAL SECTION:
d.nsset.be. 172800 IN A 194.0.44.1
d.nsset.be. 172800 IN AAAA 2001:678:6c::1
b.nsset.be. 172800 IN A 194.0.37.1
b.nsset.be. 172800 IN AAAA 2001:678:64::1
z.nsset.be. 172800 IN A 194.0.25.36
z.nsset.be. 172800 IN AAAA 2001:678:20::36
c.nsset.be. 172800 IN A 194.0.43.1
c.nsset.be. 172800 IN AAAA 2001:678:68::1
a.nsset.be. 172800 IN A 194.0.6.1
a.nsset.be. 172800 IN AAAA 2001:678:9::1
y.nsset.be. 172800 IN A 129.29.253.8
y.nsset.be. 172800 IN AAAA 2001:dcd:7::8

;; Query time: 19 msec
;; SERVER: 2001:503:b3e::2:30e53(a.root-servers.net) (TCP)
;; WHEN: Sat Oct 11 19:07:00 CEST 2025
;; MSG SIZE rcvd: 407

real 0m0.070s
user 0m0.012s
sys 0m0.003s
jeremie@pi-jeremie:~$

```

Résultats

Protocole	Query time Dig	Temps réel (real) mesuré avec time
UDP	111 ms	0,154 s
TCP	19 ms	0,070 s

Interprétation

UDP :

- Le protocole classique pour les requêtes DNS
- Léger, pas de handshake TCP
- Ici, le Query time est plus élevé (111 ms), mais le real total est plus long (0,154 s).
- Utilisation typique pour la plupart des requêtes simples

TCP :

- Plus fiable, handshake en 3 étapes mais Dig a reçu la réponse plus rapidement (Query time 19 ms, real 0,070 s)
- TCP est utile pour les réponses volumineuses ou le transfert de zone (AXFR)

- Dans ce cas précis, la réponse était courte → TCP est plus rapide que UDP → probablement dû à la gestion plus efficace du paquet par le serveur racine et à la taille du message qui dépasse la limite UDP initiale (EDNS).

Conclusion

En général, **UDP est privilégié** pour le DNS classique.

TCP est utilisé lorsque :

- La réponse dépasse ~512 octets
- Il s'agit d'un transfert de zone (AXFR)
- Une fiabilité totale est nécessaire

Dans ton test particulier avec Dig, **TCP a été plus rapide**, mais ce n'est pas le comportement typique pour toutes les requêtes DNS.